Lab Exercise: Building a Simple Counter App with React

Objective: To understand and implement state in React functional components using the `useState` hook. You will build an application with two independent counters and display the results of mathematical operations between them.

Step 1: Set Up the HTML File

First, let's create the basic HTML structure. We will include React, ReactDOM, and Babel through a CDN (Content Delivery Network). Babel is necessary to transpile our JSX code directly in the browser.

1. Create a new file and name it `index.html`.
2. Copy and paste the following boilerplate code into the file:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>React Counter Lab</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>

    <div id="root"></div>

    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>

    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

    <script type="text/babel" src="statelab.js"></script>

</body>
</html>
```

CSS

```css
body {
    font-family: sans-serif;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    background-color: #f4f4f9;
}

#root {
    width: 100%;
    max-width: 600px;
    padding: 2rem;
    background-color: white;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

h1,
h2 {
    text-align: center;
    color: #333;
}

.counters-container {
    display: flex;
    justify-content: space-around;
    margin: 2rem 0;
}

.counter-box {
    text-align: center;
    padding: 1rem;
    border: 1px solid #ddd;
    border-radius: 8px;
}

.counter-box p {
    font-size: 3rem;
    margin: 0.5rem;
    font-weight: bold;
}

.counter-box button {
    font-size: 1.5rem;
```

```
            padding: 0.5rem 1rem;
            margin: 0 0.5rem;
            cursor: pointer;
            border: 1px solid #ccc;
            background-color: #eee;
        }

        .results {
            text-align: center;
            font-size: 1.2rem;
            background-color: #eef;
            padding: 1rem;
            border-radius: 8px;
        }
```

Step 2: Create the Main App Component

Now, let's create our root component, App, and render it to the DOM.

Inside the JS file (statelb.js), add the following code:

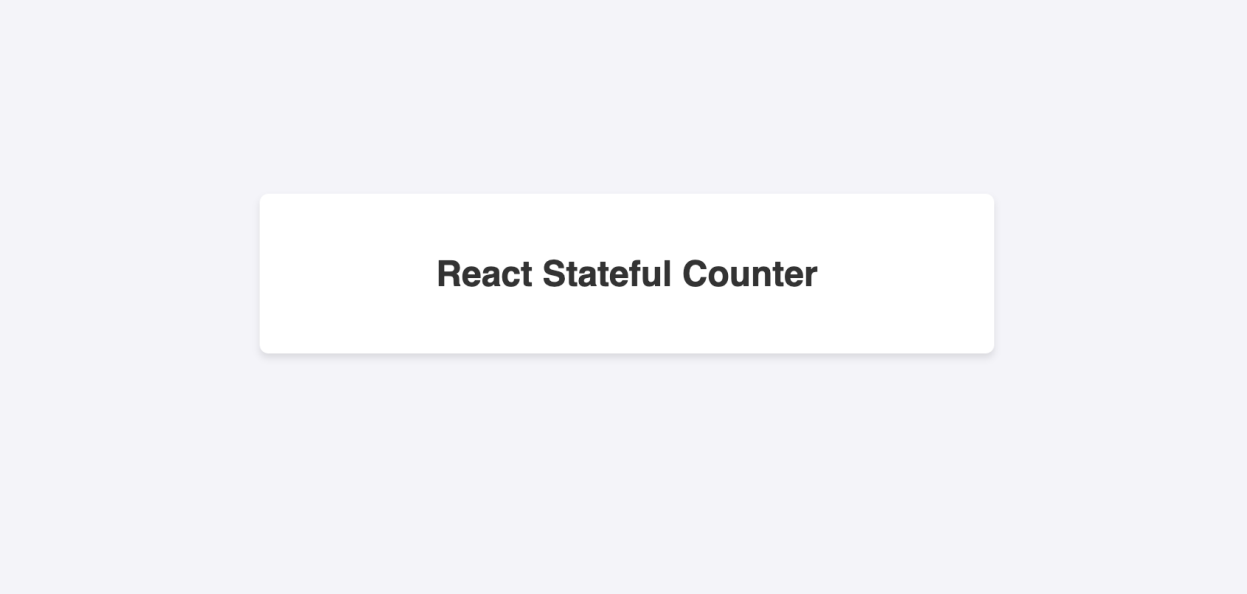
JavaScript

```
const App = () => {
    return (
        <div>
            <h1>React Stateful Counter</h1>
            {/* We will add more components here later */}
        </div>
    );
};

// --- Render the App to the DOM ---
const container = document.getElementById('root');
const root = ReactDOM.createRoot(container);
root.render(<App />);
```

Save index.html and open it in your browser. You should see the heading "React Stateful Counter".

The result shall look like:



Step 3: Introduce State with useState

The core of our app is managing the numbers, which will be our **state**. We'll use the useState hook to create two state variables in the App component, one for each counter.

1. Import useState from React.
2. Initialize two state variables, countA and countB, both starting at 0.

Update your App component like this:

JavaScript

```
// --- Import React and ReactDOM ---
const { useState } = React;

const App = () => {
    // Initialize state for two counters
    const [countA, setCountA] = useState(0);
    const [countB, setCountB] = useState(1); // Let's start the second one at 1

    return (
        <div>
            <h1>React Stateful Counter</h1>
            {/* We will add more components here later */}
        </div>
    );
};
```

- `useState(0)` initializes a piece of state with a default value of `0`.
- It returns an array containing two elements:
    1. The current state value (`countA`).
    2. A function to update that value (`setCountA`).
- We added a `results` section to display the calculations in real-time. Notice the check for division by zero.

Save and refresh your browser. You should now see the calculations based on the initial state values.

Step 4: Create a Reusable `CounterSection` Component

To avoid repeating code, we'll create a reusable component for the counter display and its buttons. This component will be "dumb"—it will only display data and report back when a button is clicked. It will receive all its data and functions as **props**.

Add this new component **above** the `App` component:

JavaScript

```
// Counter Section Component
const CounterSection = ({ label, count, setCount }) => {
    return (
        <div className="counter-section">
         <h3>{label}: {count}</h3>
         <button onClick={() => setCount(count + 1)}>+ Increment {label.split(' ')[1]}</button>
         <button onClick={() => setCount(count - 1)}>- Decrement {label.split(' ')[1]}</button>
        </div>
    );
};
```

- This component takes four props: a `label` (like "Counter A"), the `count` to display, and two functions (`onIncrease`, `onDecrease`) to call when the buttons are clicked.

Add the following to your CSS file for styling the CounterSection elements:

```
.controls {
            display: flex;
            justify-content: space-around;
            margin: 2rem 0;
            gap: 2rem;
        }

        .counter-section {
            text-align: center;
            padding: 1rem;
            border: 1px solid #ddd;
            border-radius: 8px;
            background-color: #f9f9f9;
        }

        .counter-section h3 {
            margin: 0 0 1rem 0;
            color: #555;
            font-size: 1.5rem;
        }

        .counter-section button {
            font-size: 1rem;
            padding: 0.5rem 1rem;
            margin: 0.25rem;
            cursor: pointer;
            border: 1px solid #ccc;
            background-color: #fff;
            border-radius: 4px;
            transition: background-color 0.2s;
        }

        .counter-section button:hover {
            background-color: #e0e0e0;
        }
```

Step 5: Create a Reusable Results Component

To avoid repeating code, we'll create a reusable component for the results display. This component shows the math operations:

```
// Results Component
const Results = ({ countA, countB }) => {
    return (
        <div className="results">
            <h2>Calculations</h2>
            <p>{countA} + {countB} = {countA + countB}</p>
            <p>{countA} - {countB} = {countA - countB}</p>
            <p>{countA} * {countB} = {countA * countB}</p>
            <p>{countA} / {countB} = {countB !== 0 ? (countA / countB).toFixed(2) :
'Cannot divide by zero'}</p>
        </div>
    );
};
```

Step 6: Connect Everything Together

The final step is to use our new Counter component inside the App component and pass it the state values and functions to update the state.

Update the App component to include the two Counter instances:

JavaScript

```
const { useState } = React;

// Counter Section Component
const CounterSection = ({ label, count, setCount }) => {
    return (
        <div className="counter-section">
            <h3>{label}: {count}</h3>
            <button onClick={() => setCount(count + 1)}>+ Increment {label.split(' ')[1]}</button>
            <button onClick={() => setCount(count - 1)}>- Decrement {label.split(' ')[1]}</button>
        </div>
    );
};

// Results Component
const Results = ({ countA, countB }) => {
    return (
        <div className="results">
            <h2>Calculations</h2>
            <p>{countA} + {countB} = {countA + countB}</p>
            <p>{countA} - {countB} = {countA - countB}</p>
            <p>{countA} * {countB} = {countA * countB}</p>
            <p>{countA} / {countB} = {countB !== 0 ? (countA / countB).toFixed(2) : 'Cannot divide by
zero'}</p>
        </div>
    );
};

const App = () => {
    // Initialize state for two counters
    const [countA, setCountA] = useState(0);
    const [countB, setCountB] = useState(1); // Let's start the second one at 1

    return (
        <div>
            <h1>React Stateful Counter</h1>

            <div className="controls">
```

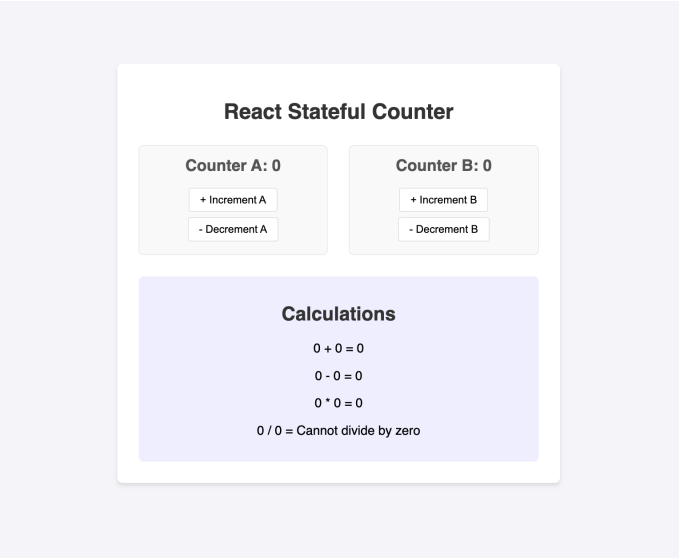
```
                <CounterSection
                    label="Counter A"
                    count={countA}
                    setCount={setCountA}
                />
                <CounterSection
                    label="Counter B"
                    count={countB}
                    setCount={setCountB}
                />
            </div>

            <Results countA={countA} countB={countB} />
        </div>
    );
};
```

Explanation:

- **count={countA}**: We pass the current state value countA, and countB down to the Counter component.
- **onInclick={() => setCountA(count + 1)}**: We pass an anonymous function as a prop. When the + button inside the Counter is clicked, this function is executed, which calls setCountA to update the state in the parent App component.
- When the state is updated, React automatically re-renders the App component and its children (Counter and the results div) with the new values. This is the core of React's declarative nature.

Page will look like:



Challenge

Add a "Reset" button to each counter box that sets its specific count back to `0`. Then, add a global "Reset All" button at the top of the page that resets both counters simultaneously.